Exercices - Corrections - Structures hiérarchiques : arbres

Exercice 1

1. Un **arbre binaire** est un arbre dont tous les noeuds ont **au plus** deux fils.
2. La **taille** d'un arbre est le nombre de noeuds qu'il possède.
3. La **profondeur** d'un noeud est la longueur du chemin le plus court entre ce noeud et la racine (la racine a donc une profondeur égale à 0).
4. Si on note H la hauteur d'un arbre binaire à 12 noeuds, alors :

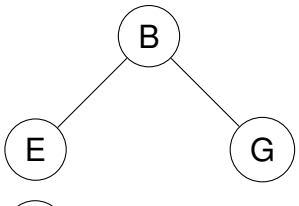
$$E(\log_2 12) \leq H \leq 12 - 1$$

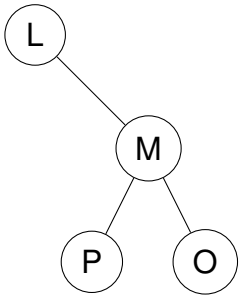
$\Leftrightarrow 3 \leq H \leq 11$

Exercice 2

1. **Arbre 1**
   - Oui, car tous les noeuds possèdent au plus deux fils.
   - A
   - D ; E ; F
   - taille : 6
   - hauteur : 2
   - profondeur D : 2
   -

   - Arbre vide
2. **Arbre 2**
   - Non car le noeud G possède 3 fils.
   - A
   - D ; E ; H ; I ; J
   - taille : 9
   - hauteur : 3
   - profondeur D : 3
3. **Arbre 3**
   - Oui, car tous les noeuds possèdent au plus deux fils.
   - A
   - B ; D ; F ; G
   - taille : 7
   - hauteur : 3
   - profondeur D : 2
   -

   - Arbre vide
4. **Arbre 4**
   - Oui, car tous les noeuds possèdent au plus deux fils.
   - A
   - E ; G ; I ; J ; N ; P ; O
   - taille : 15
   - hauteur : 4
   - profondeur D : 2
   - 

   - 

Exercice 3

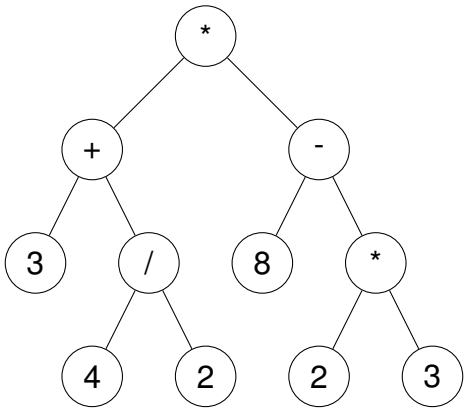
Reprendre les arbres 1, 3 et 4 de l'exercice 2 et donner la liste des étiquettes selon les algorithmes de parcours :

1. Arbre 1
   - largeur d'abord : A ; B ; C ;D ; E ; F
   - pronfondeur infixe : D ; B ; E ; A ; F ; C
   - profondeur prefixe : A ; B ; D ; E ; C ; F
   - profondeur suffixe : D ; E ; B ;F ; C ; A
2. Arbre 3
   - largeur d'abord : A ; B ; C ; D ; E ; F ; G
   - pronfondeur infixe : B ; A ; D ; C ; F ; E ; G
   - profondeur prefixe : A ; B ; C ; D ; F ; E ; G
   - profondeur suffixe : B ; D ; F ; G ; E ; C ; A
3. Arbre 4
   - largeur d'abord : A ; C ; F ; B ; D ; K ; L ; E ;G ; I ; J ; N ; M ; P ; O
   - pronfondeur infixe : E ; B ; G ; C ; I ; D ; J ; A ; K ; N ; F ; L ; P ; M ; O
   - profondeur prefixe : A ; C ; B ; E ; G ; D ; I ; J ; F ; K ; N ; L ; M ; P ; O
   - profondeur suffixe : E ; G ; B ; I ; J ; D ; C ; N ; K ; P ; O ; M ; L ; F ; A

Exercice 4

On donne ci-contre le tableau caractérisant un arbre :

1. Représenter l'arbre correspondant.



Nœud	Etiquette	Nœud du SAG	Nœud du SAD
1	*	2	3
2	+	4	5
3	-	6	7
4	3		
5	/	8	9
6	8		
7	*	10	11
8	4		
9	2		
10	2		
11	3		

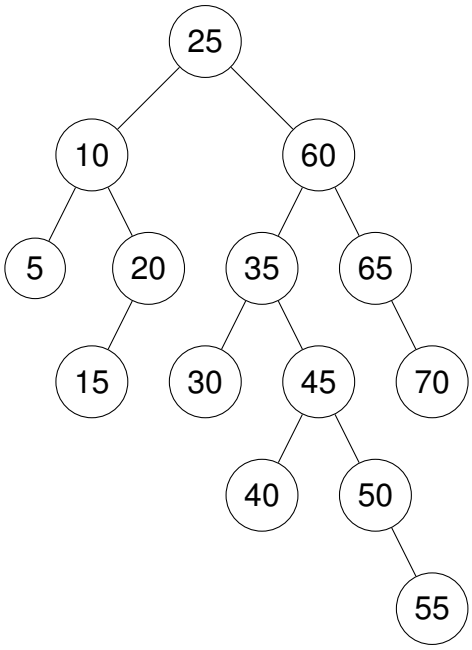
2. hauteur : 3
3. taille : 11
4. oui, c'est un arbre binaire (au plus 2 fils par noeuds)
5. Non
6. parcours infixe : 3 + 4 / 2 * 8 - 2 * 3
7. 13

Exercice 5

1. Un **arbre binaire de recherche**, abrégé **ABR**, est un arbre binaire dans lequel tout noeud a une clé (= étiquette) :

- plus grande ou égale à celles de son sous-arbre gauche
- plus petite strictement que celles de son sous-arbre droit

2. On trouve le minimum en se déplaçant systématiquement à gauche : le dernier noeud atteint avant un sous-arbre gauche vide est le minimum ;
3. parcours profondeur infixe.
4. On donne ci-dessous une liste aléatoire de 14 nombres entiers :
   25 ; 60 ; 35 ; 10 ; 5 ; 20 ; 65 ; 45 ; 70 ; 40 ; 50 ; 55 ; 30 ; 15



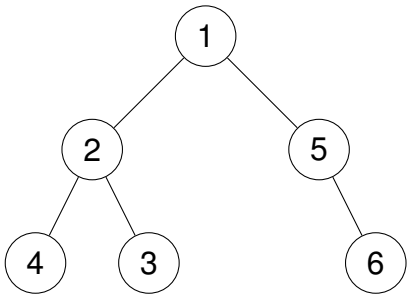
5. On veut chercher l'étiquette "70". Expliquer comment procéder en partant de la racine.
   - $70 > 35$, donc on cherche dans le sous arbre droit de racine 60.
   - $70 > 60$, donc on cherche dans le sous arbre droit de racine 65.
   - $70 > 65$, donc on cherche dans le sous arbre droit de racine 70.
   - $70 = 70$, la valeur recherchée est égale à la racine, on a trouvé l'étiquette.
6. Même question pour chercher l'étiquette "5".
   - $5 < 25$, donc on cherche dans le sous arbre gauche de racine 10.
   - $5 < 10$, donc on cherche dans le sous arbre droit de racine 5.
   - $5 = 5$, la valeur recherchée est égale à la racine, on a trouvé l'étiquette.
7. Même question pour chercher l'étiquette "150".
   - $150 > 35$, donc on cherche dans le sous arbre droit de racine 60.
   - $150 > 60$, donc on cherche dans le sous arbre droit de racine 65.
   - $150 > 65$, donc on cherche dans le sous arbre droit de racine 70.
   - $150 > 70$, donc on cherche dans le sous arbre droit qui est vide.
   - l'arbre est vide, l'étiquette recherchée n'est pas dans l'arbre.

Exercice 6

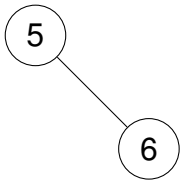
On donne la suite d'instructions suivantes :

```
A = CREER_ARBRE(2,CREER_ARBRE_FEUILLE(4), CREER_ARBRE_FEUILLE(3))
B = CREER_ARBRE(5,CREER_ARBRE_VIDE(), CREER_ARBRE_FEUILLE(6))
C = CREER_ARBRE(1, A, B)
```

1. Arbre C



2. Donner l'arbre correspondant à l'instruction : `T = SAD(C)`



3. `r = 5)`

Exercice 7

Ci-dessous une implémentation Python d'une classe `Noeud` :

```
class Noeud:
    def __init__(self, e, g=None, d=None):
        self.etiquette = e
        self.gauche = g
        self.droit = d

    def est_feuille(self):
        return not self.gauche and not self.droit

    # Une representation possible de l'arbre
    def __repr__(self):
        ch = str(self.etiquette)
        if self.gauche or self.droit:
            ch = ch + '-(' + str(self.gauche) + ',' + str(self.droit) + ')'
        return ch
```

1. Ecrire les instructions python permettant de créer les arbre 1 et 3 de l'exercice 2.

```
A1 = Noeud('A', Noeud('B', Noeud('D'), Noeud('E')), Noeud('C', Noeud('F'), None
    ))
A3 = Noeud('A', Noeud('B'), Noeud('C', Noeud('D'), Noeud('E',Noeud('F'),Noeud('
    G'))))
```

2. Ecrire une fonction python `taille`, prenant en paramètre un objet `a` de type `Noeud`, qui renvoie la taille de l'arbre `a`.

```
def taille(a):
    """Renvoie la taille d'un arbre binaire a."""
    if a is None:
        return 0
    else:
        return 1 + taille(a.gauche) + taille(a.droit)
```

3. Ecrire une fonction python `parcours_suffixe`, prenant en paramètre un objet `a` de type `Noeud`, qui affiche la liste des étiquettes de l'arbre `a` selon le parcours en profondeur suffixe.

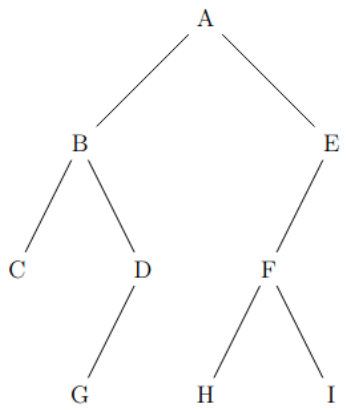
```
def parcours_suffixe(a):
    """Affiche les etiquettes de l'arbre binaire a selon un parcours suffixe
        ."""
    if a is not None:
        parcours_prefixe(a.gauche)
        parcours_prefixe(a.droit)
        print(a.etiquette, end=" ")  # etiquette affichee apres SAG et SAD
```

4. Ecrire une fonction python `recherche`, prenant en paramètre un objet `a` de type `Noeud` et un entier n, qui retourne `True` si la valeur n appartient à l'arbre `a` et `False` sinon.
Nous faisons l'hypothèse que l'arbre `a` est un **arbre binaire de recherche**.

```
def etq_presente(a, n):
    """Renvoie True si l'etiquette n est presente dans l'ABR a, et False sinon
        ."""
    if a is None:
        return False
    if n == a.etiquette:
        return True
    elif n < a.etiquette:
        return etq_presente(a.gauche, n)
    else:
        return etq_presente(a.droit, n)
```

Exercice 8 Extrait BAC NSI

Dans cet exercice, on utilisera la convention suivante : la hauteur d'un arbre binaire ne comportant qu'un noeud est 1.



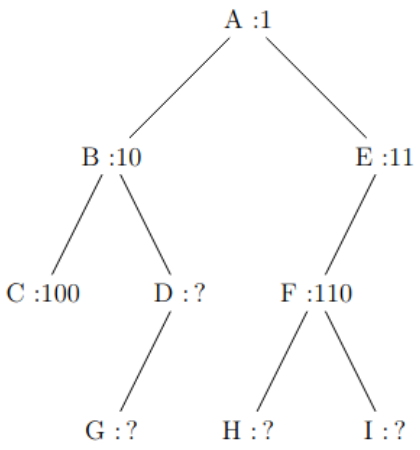
Question 1
Taille = 9 ; hauteur = 4

Question 2
On décide de numéroter en binaire les noeuds d'un arbre binaire de la façon suivante :

- la racine correspond à 1 ;
- la numérotation pour un fils gauche s'obtient en ajoutant le chiffre 0 à droite au numéro de son père ;
- la numérotation pour un fils droit s'obtient en ajoutant le chiffre 1 à droite au numéro de son père ;

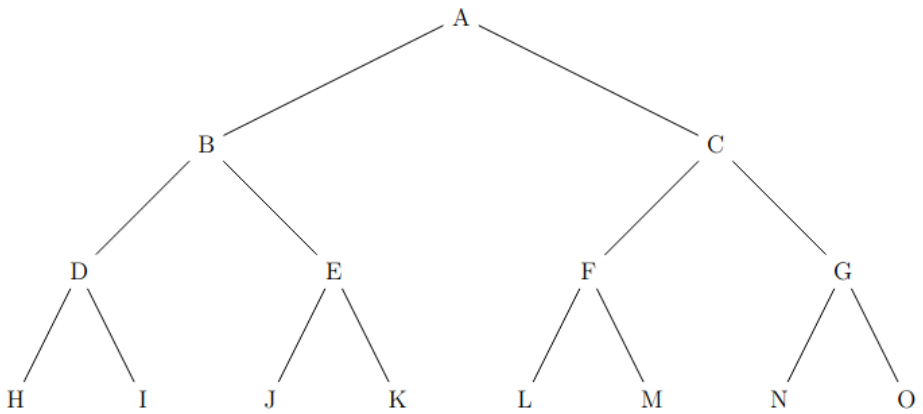
Par exemple, dans l'arbre ci-dessous, on a utilisé ce procédé pour numéroter les nœuds A, B, C, E et F .

1. G : 1010
2. I : 1101
3. A chaque “étage”, on augmente le nombre de bits de 1 : si h = 1, nombre de bit = 1 ; si h = 2, nombre de bits = 2... pour une hauteur h le nombre de bits est de h.
4. Dans le cas de l’arbre filiforme nous avons, pour $h = 3$, $n = 3$ (n : taille). Si on généralise pour un arbre de hauteur h, nous avons $n = h$.
   Dans le cas d’un arbre complet, pour $h = 3$ nous avons $n = 7$, donc $n = 2^3 - 1 = 7$.
   Si on généralise pour un arbre de hauteur h, nous avons $n = 2^h - 1$. Sachant qu’un arbre quelconque est un intermédiaire entre l’arbre filiforme et l’arbre complet, nous pouvons donc dire que $h \leq n \leq 2^h - 1$.



Question 3

Un arbre binaire est dit complet si tous les niveaux de l’arbre sont remplis.



Arbre binaire complet

On décide de représenter un arbre binaire complet par un tableau de taille $n + 1$, où n est la taille de l’arbre, de la façon suivante :

- La racine a pour indice 1 ;
- Le fils gauche du nœud d’indice i a pour indice $2 \times i$;
- Le fils droit du nœud d’indice i a pour indice $2 \times i + 1$;
- On place la taille n de l’arbre dans la case d’indice 0.

1. ”[15, ‘A’, ‘B’, ‘C’, ‘D’, ‘E’, ‘F’, ‘G’, ‘H’, ‘I’, ‘J’, ‘K’, ‘L’, ‘M’, ‘N’, ‘O’]”
2. indice du père : `k = i // 2`

CC BY NC SA